

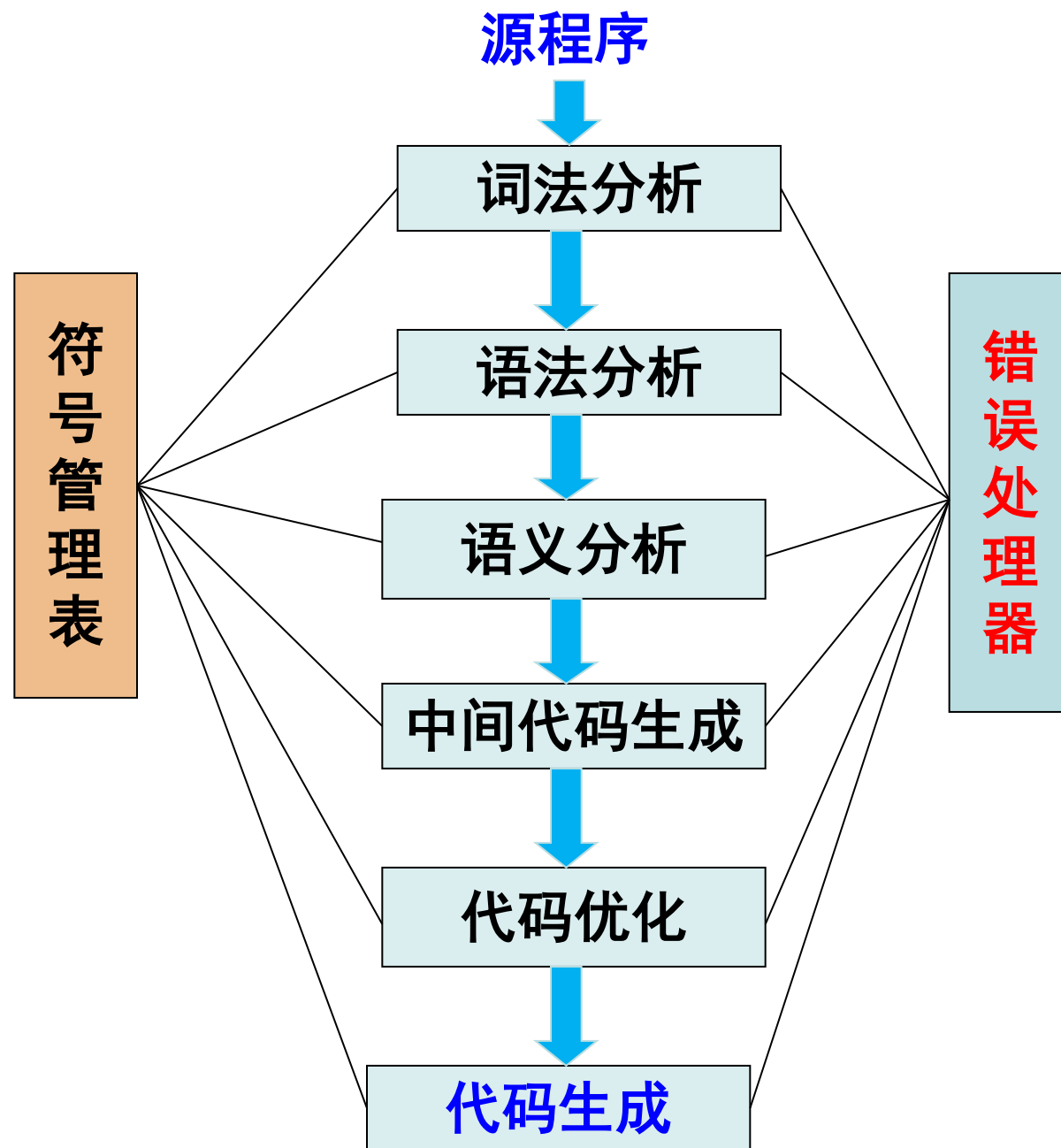


信息与软件工程学院

编译技术

主讲教师： 陈安龙

上节知识回顾



第2章 词法分析器

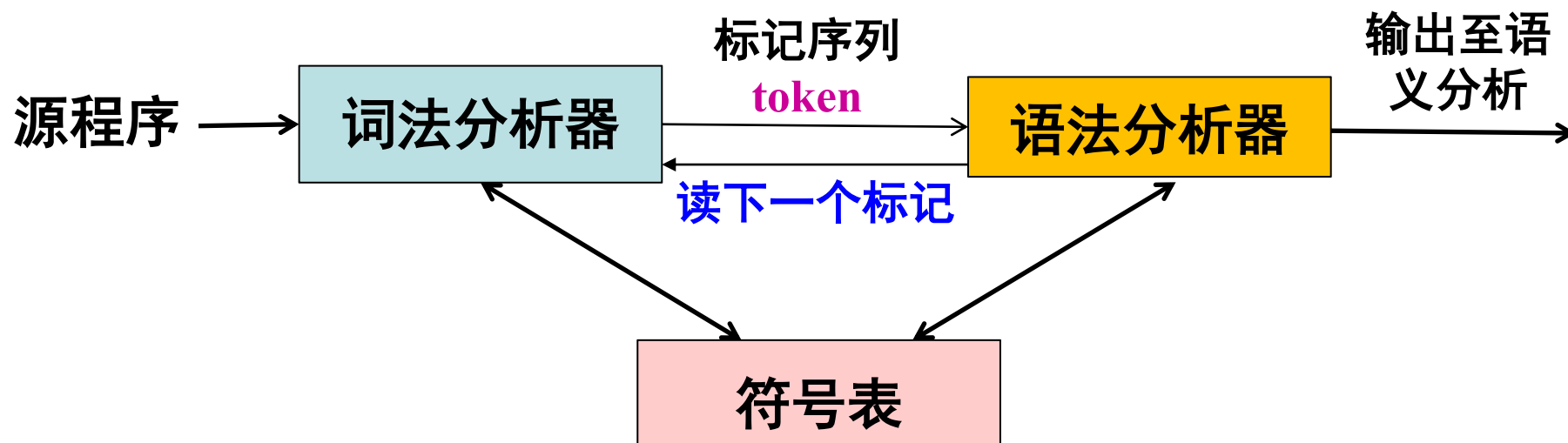
- 词法分析器简介
- 词法分析器的实现方法
- 正则表达式
- 状态自动机NFA、DFA
- Thompson构造算法
- DFA的优化：Hopcroft算法
- 简单词法分析器示例

1. 词法分析总体介绍

- 编译技术是计算机科学中一门重要的领域，它涉及将高级编程语言转换为机器可执行代码的过程。在编译器的构建过程中，词法分析是一个至关重要的步骤，它负责将源代码转换为标记序列，为后续的语法分析和语义分析阶段提供输入。
- 词法分析（Lexical Analysis）词法分析器的主要任务是识别源代码中的各种词法单元，词法单元是源程序代码中的最小语法单位，如关键字、标识符、常量、运算符等，并将它们转换为标记(token)。

词法分析总体介绍

- 词法分析的结果是一个标记序列，它将源代码中的每个词法单元映射到一个标记（token），这些标记将作为语法分析器的输入。通过词法分析，编译器能够有效地处理源代码，识别其中的结构和语义，并为后续的编译过程奠定基础。



词法单元

- 词法单元是编程语言中的基本单元，是词法分析的结果。它是由词法分析器从源程序中识别并返回的具有独立含义的最小代码单元（单词）。
- 例如：在C语言中，词法单元可以是关键字（如int、if）、标识符（如变量名）、常量（如数字常量）、运算符（如+、-）等。

词法分析器的相关技术

- 正则表达式：

- ① 描述词法单元模式：正则表达式被广泛用于描述词法单元的模式，如标识符、数字、运算符等。通过正则表达式，可以定义词法单元的识别规则。
- ② 匹配模式：词法分析器使用正则表达式来匹配源代码中的词法单元，从而识别并提取标记。

词法分析器的相关技术

- 有限状态自动机（Finite State Automaton）：

- ① 描述词法单元的识别过程：有限状态自动机被用来描述词法单元的识别过程。每个状态代表词法分析器在识别过程中的状态，状态之间的转移表示词法单元的转换过程。
- ② 状态转移表：词法分析器通常使用状态转移表来存储状态之间的转移规则，以实现词法单元的识别。

词法分析器的相关技术

- 词法分析器生成器：

- ① 自动生成词法分析器代码：词法分析器生成器（如FLex工具）可以根据用户定义的正则表达式和词法规则自动生成词法分析器的代码。
- ② 简化词法分析器的实现：词法分析器生成器可以简化词法分析器的实现过程，提高开发效率。

词法分析器的相关技术

- 标记生成：

- ① 生成标记序列：词法分析器识别词法单元后，将其转换为对应的标记，并生成标记序列作为语法分析器的输入。
- ② 支持后续分析：生成的标记序列为后续的语法分析和语义分析提供了必要的输入，帮助编译器理解源代码的结构和含义。

词法分析器生成标记序列的一般过程：

- ① **识别词法单元**：词法分析器从输入缓冲区逐个读取字符，并根据词法规则和状态转移表识别词法单元，如标识符、关键字、运算符等。
- ② **生成标记**：当词法单元被识别时，词法分析器将其转换为对应的标记。每个标记通常包含两部分信息：词法单元的类型（如标识符，关键字）和对应的属性值（如标识符的名称）。
- ③ **构建标记序列**：词法分析器将生成的标记按顺序组成标记序列，作为语法分析器的输入。标记序列将帮助后续的语法分析和语义分析阶段理解源代码的结构和含义。

标记序列通常使用数据结构来存储，常见的数据结构包括：

- ① **Token类**：表示一个标记，包含词法单元的类型和属性值。
- ② **Token序列**：由多个Token组成的序列，用于表示整个源代码的标记序列。

以下是一个简单的C语言代码片段：

```
int main() {  
    int a = 10;  
    float b = 3.14;  
    printf("Sum: %f\n", a + b);  
    return 0;  
}
```

识别词法单元：

关键字：int、float、return

标识符：main、a、b、printf

运算符：=、+

分界符：<、>、(、)、;、,

整数常量：10

浮点数常量：3.14

字符串常量："Sum: %f\n"

构建标记(token)序列：

```
<int, > <id,main> <(, > <), > <{, >  
<int, > <id,a> <assign_op,=> <number,10>  
<float, > <id,b> <assign_op,=> <number,3.14>  
<id,printf> <(, > <string,"Sum: %f\n">  
<id,a> <op,+> <id,b> <), > <;, >  
<return, > <number,0> <;, >  
<}, >
```

注意：标记符的第1部分是词法单元名或类别，第2部分是词法单元的属性值，或者是指向符号表中该词法单元的指针。

在某些编译器中，为了方便实现，将词法单元的单词按类别进行编码，在标记序列中记录类别编码，属性中记录指向符号表记录的指针。每个关键字、分界符和运算符单独编码，对应的属性值为词法单元符号本身，常数对应属性值为常数值本身。

类别	单词	种别码	类别	单词	种别码	类别	单词	种别码
关键字	program	1	关键字	write	18	标识符	id	34
	var	2		true	19			
	integer	3		false	20			
	bool	4	运算符	not	21	常数	整常数	35
	real	5		and	22		实常数	36
	char	6		or	23		字符常数	37
	const	7		+	24		布尔常数	38
	begin	8		-	25	分界符	=	39
	if	9		*	26		;	40
	then	10		/	27		,	41
	else	11		<	28		'	42
	while	12		>	29		/*	43
	do	13		<=	30		*/	44
	for	14		>=	31		:	45
	to	15		==	32		(	46
	end	16		<>	33		)	47
	read	17					.	48

■ 例如有程序语句：

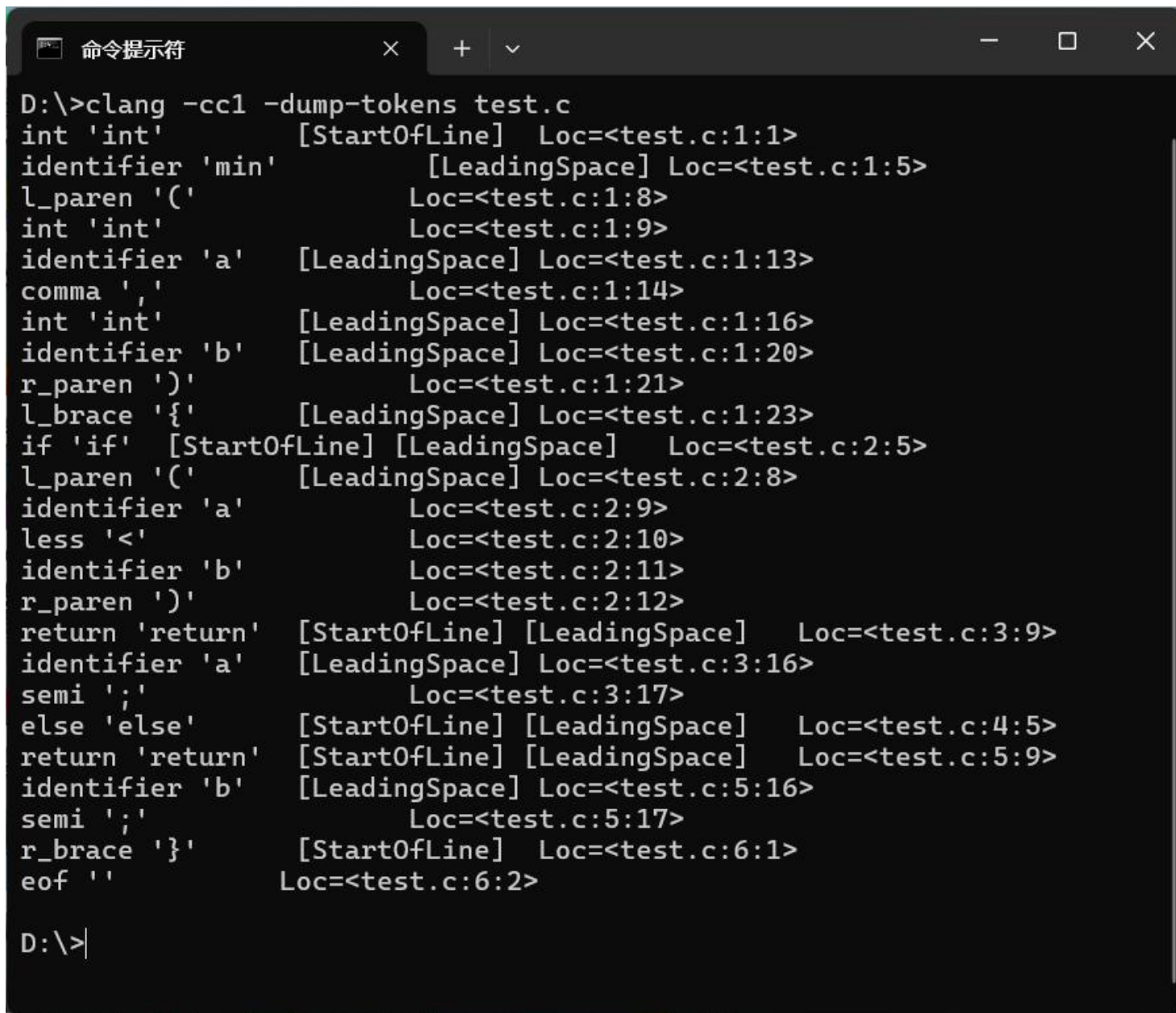
```
if i>j then  
    i=0  
else j=1
```

请注意：具体实现方式
取决编译器设计者的整
体考虑。

if	<9, if >
i	<34, 符号表记录地址>
>	<29, > >
j	<34, 符号表记录地址>
then	<10, then >
i	<34, 符号表记录地址>
=	<39, = >
0	<35, 0 >
else	<11, else >
j	<34, 符号表记录地址>
=	<39, = >
1	<35, 1 >

具体编译器采用的token结构有所不同，右边面是llvm/clang经词法分析器输出的token流。

```
int min(int a, int b) {  
    if (a<b)  
        return a;  
    else  
        return b;  
}
```



```
D:\>clang -cc1 -dump-tokens test.c  
int 'int' [StartOfLine] Loc=<test.c:1:1>  
identifier 'min' [LeadingSpace] Loc=<test.c:1:5>  
l_paren '(' Loc=<test.c:1:8>  
int 'int' Loc=<test.c:1:9>  
identifier 'a' [LeadingSpace] Loc=<test.c:1:13>  
comma ',' Loc=<test.c:1:14>  
int 'int' [LeadingSpace] Loc=<test.c:1:16>  
identifier 'b' [LeadingSpace] Loc=<test.c:1:20>  
r_paren ')' Loc=<test.c:1:21>  
l_brace '{' [LeadingSpace] Loc=<test.c:1:23>  
if 'if' [StartOfLine] [LeadingSpace] Loc=<test.c:2:5>  
l_paren '(' [LeadingSpace] Loc=<test.c:2:8>  
identifier 'a' Loc=<test.c:2:9>  
less '<' Loc=<test.c:2:10>  
identifier 'b' Loc=<test.c:2:11>  
r_paren ')' Loc=<test.c:2:12>  
return 'return' [StartOfLine] [LeadingSpace] Loc=<test.c:3:9>  
identifier 'a' [LeadingSpace] Loc=<test.c:3:16>  
semi ';' Loc=<test.c:3:17>  
else 'else' [StartOfLine] [LeadingSpace] Loc=<test.c:4:5>  
return 'return' [StartOfLine] [LeadingSpace] Loc=<test.c:5:9>  
identifier 'b' [LeadingSpace] Loc=<test.c:5:16>  
semi ';' Loc=<test.c:5:17>  
r_brace '}' [StartOfLine] Loc=<test.c:6:1>  
eof '' Loc=<test.c:6:2>  
  
D:\>
```


如何表达正确的词法模式？

- 编译器通过使用正则表达式或有限自动机（Finite Automaton）来表达正确的词法结构模式，描述了词法单元（tokens）的形式，例如标识符、关键字、常量、运算符和界符等。

正则表达式

- 正则表达式是一种**用于描述字符串模式的形式语言**，它可以精确地匹配文本中的**特定模式**。正则表达式由**普通字符（如字母、数字）和特殊字符（如元字符）**组成，可以用来进行字符串匹配、查找、替换等操作。在编译器中，正则表达式通常用于词法分析阶段，**用于定义词法单元（tokens）的模式**。
- 正则表达式的准确定义指的是**可以通过有限步骤确定性地判断一个字符串是否符合该正则表达式的模式**。换句话说，正则表达式的匹配过程是可预测且有限的，**不会出现歧义或无限循环的情况**。

正则表达式的归纳描述

- ① ε 是表示空字符的正则表达式, $L(\varepsilon) = \{\varepsilon\}$ 表示只含空串的语言;
- ② 如果 a 是集 Σ 上的一个符号, 那么 a 是正则表达式, 并且 $L(a) = \{a\}$ 表示仅含有字符串 a 的语言;
- ③ 假设 r 和 s 都是正则表达式, 分别表示的语言为 $L(r)$ 和 $L(s)$, 那么
 - (a) $(r) | (s)$ 是一个正则表达式, 表示语言 $L(r) \cup L(s)$
 - (b) $(r)(s)$ 是一个正则表达式, 表示语言 $L(r) L(s)$
 - (c) $(r)^*$ 是一个正则表达式, 表示语言 $(L(r))^*$, $*$ 表示由0个或多个;
 - (d) $(r)^+$ 是一个正则表达式, 表示语言 $L(r)^+$

有时可以括号, $((a)(b)^*) | (c)$ 可以简写为 $ab^* | c$

正则表达式的例子 字母表 $\Sigma = \{a, b\}$

- 正则表达式 表示的字符串的集合(或者语言)

$a \mid b$ $\{a, b\}$

$(a \mid b)(a \mid b)$ $\{aa, ab, ba, bb\}$

$aa \mid ab \mid ba \mid bb$ $\{aa, ab, ba, bb\}$

a^* 由0个或多个 a 构成的所有字符串集合

$(a \mid b)^*$ 由任意个 a 或 b 构成的字符串集合

$a \mid a^*b$

- 复杂的例子

$(00 \mid 11 \mid ((01 \mid 10)(00 \mid 11)^*(01 \mid 10)))^*$

偶数个0和偶数个1组成的01字符串

正则表达式的代数性质

定律	描述
$r \mid s = s \mid r$	$\mid$ 是可交换的
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ 是可结合的
$(r \mid s) t = r (s t)$	“连接” 是可结合的
$r (s \mid t) = r s \mid r t$ $(s \mid t) r = s r \mid t r$	“连接” 是可分配的
$\varepsilon r = r$ $r \varepsilon = r$	ε 是 “连接” 的单位元
$r^* = (r \mid \varepsilon)^*$	$*$ 和 ε 之间的关系
$r^{**} = r^*$	$*$ 的幂等性

$$(rs)^* \neq r^*s^*$$
$$(r|s)^* \neq r^*|s^*$$

正则表达式的定义式

为了让正则表达式的表示简洁，可以对正则表达式取一个名字。

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

每个 d_i 就是正则表达式的名字,并且互不相同 $d_i \notin \Sigma$

每个 r_i 都是 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则式

正则定义的例子1

C语言的标识符集合

$letter \rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _$

$digit \rightarrow 0 \mid 1 \mid \dots \mid 9$

$id \rightarrow letter (letter \mid digit)^*$

如果代入，则有C语言标识符的正则表达式为

$(A \mid \dots \mid Z \mid a \mid \dots \mid z \mid _)(A \mid \dots \mid Z \mid a \mid \dots \mid z \mid _ \mid 0 \mid 1 \mid \dots \mid 9)^*$

正则定义的例子2

无符号数的正则表达式, 例如: 1946, 11.28, 63.6E8, 1.99E-6

digit $\rightarrow 0 \mid 1 \mid \dots \mid 9$

digits $\rightarrow digit\ digit^*$

optionalFraction $\rightarrow .digits \mid \epsilon$

optionalExponent $\rightarrow (E\ (+\mid-\mid\epsilon)\ digits) \mid \epsilon$

number $\rightarrow digits\ optionalFraction\ optionalExponent$

正则表达式的扩展

1. “+”：一个或多个，表示一个正则表达式及其语言的正闭包， r^+ 表示语言 $(L(r))^+$

$$r^* = r^+ \mid \varepsilon$$

$$r^+ = r r^*$$

2. “?”：零个或一个出现，也就是说， $r?$ 等价于 $r \mid \varepsilon$

3. “[]”：一个正则表达式 $a_1 \mid a_2 \mid \dots \mid a_n$ （其中 a_i 是字母表中的各个符号）可以缩写成

$[a_1 a_2 \dots a_n]$ 。

例如： $[ade] = a \mid d \mid e$

[范围]：如果 $a_1 \mid a_2 \mid \dots \mid a_n$ 是连续字符集、数字集等，

例如： $[A-Z] = A \mid B \mid C \mid \dots \mid Z$

正则表达式的扩展

1. “+”：一个或多个，表示一个正则表达式及其语言的正闭包， r^+ 表示语言 $(L(r))^+$

$$r^* = r^+ \mid \varepsilon$$

$$r^+ = r r^*$$

2. “?”：零个或一个出现，也就是说， $r?$ 等价于 $r \mid \varepsilon$

3. “[]”：一个正则表达式 $a_1 \mid a_2 \mid \dots \mid a_n$ （其中 a_i 是字母表中的各个符号）可以缩写成

$[a_1 a_2 \dots a_n]$ 。

例如： $[ade] = a \mid d \mid e$

[范围]：如果 $a_1 \mid a_2 \mid \dots \mid a_n$ 是连续字符集、数字集等，

例如： $[A-Z] = A \mid B \mid C \mid \dots \mid Z$

使用简化的表达方式

- C语言的标识符集合

$letter \rightarrow A | B | \dots | Z | a | b | \dots | z | _$

$digit \rightarrow 0 | 1 | \dots | 9$

$id \rightarrow letter (letter | digit)^*$

- 简化为：

$letter \rightarrow [A-Za-z_]$

$digit \rightarrow [0-9]$

$id \rightarrow letter (letter | digit)^*$

则C语言标识符的正则表达式： $[A-Za-z_]([A-Za-z_] | [0-9])^*$

无符号数的正则表达式

$digit \rightarrow [0-9]$

$digits \rightarrow digit^+$

$number \rightarrow digits (.digits)? (E(+|-)? digits)?$

则无符号数的正则表达式为: $[0-9]^+ ([0-9]^+)? (E(+|-)? [0-9]^+)?$

给出 $\Sigma=\{0, 1\}$ 上的正则表达式

① 仅含有两个1，任意个0

$0^*10^*10^*$

② 不包含连续的0

$(1 \mid 01)^*(\varepsilon \mid 0)$

③ 包含偶数个0

$(1^*01^*01^*)^*1^*$

请思考下列问题：

假设给定写出如下正则表达式，如果要写出识别某字符串是否符合该正则表达式，请思考如何用编程语言来实现！

- ① C语言标识符的正则表达式： $[A-Za-z_]([A-Za-z_] | [0-9])^*$
- ② 无符号数的正则表达式为： $[0-9]^+ (.[0-9]^+)? (E(+|-)? [0-9]^+)?$

写出能正确分析识别字符串是否满足词法规则的分析程序是比较困难的？

采用什么方法可以降低设计这样程序的难度呢？

有限状态自动机

正则表达式的练习题

习题1： 写出正则表达式： 匹配所有以字母开头，后跟任意个数字或字母的字符串。

习题2： 写出正则表达式： 匹配所有由连续的 0 和 1 组成的字符串。

习题3： 写出正则表达式： 匹配所有**仅由偶数个 1** 组成的字符串。

习题4： 写出正则表达式： 匹配所有**仅由奇数个 1** 组成的字符串。

习题5： 写出正则表达式： 匹配所有由 a、b、c 三个字符组成的字符串，且每个字符至少出现一次。

习题6： 正则表达式： 匹配所有由至少一个数字和一个字母组成的字符串。

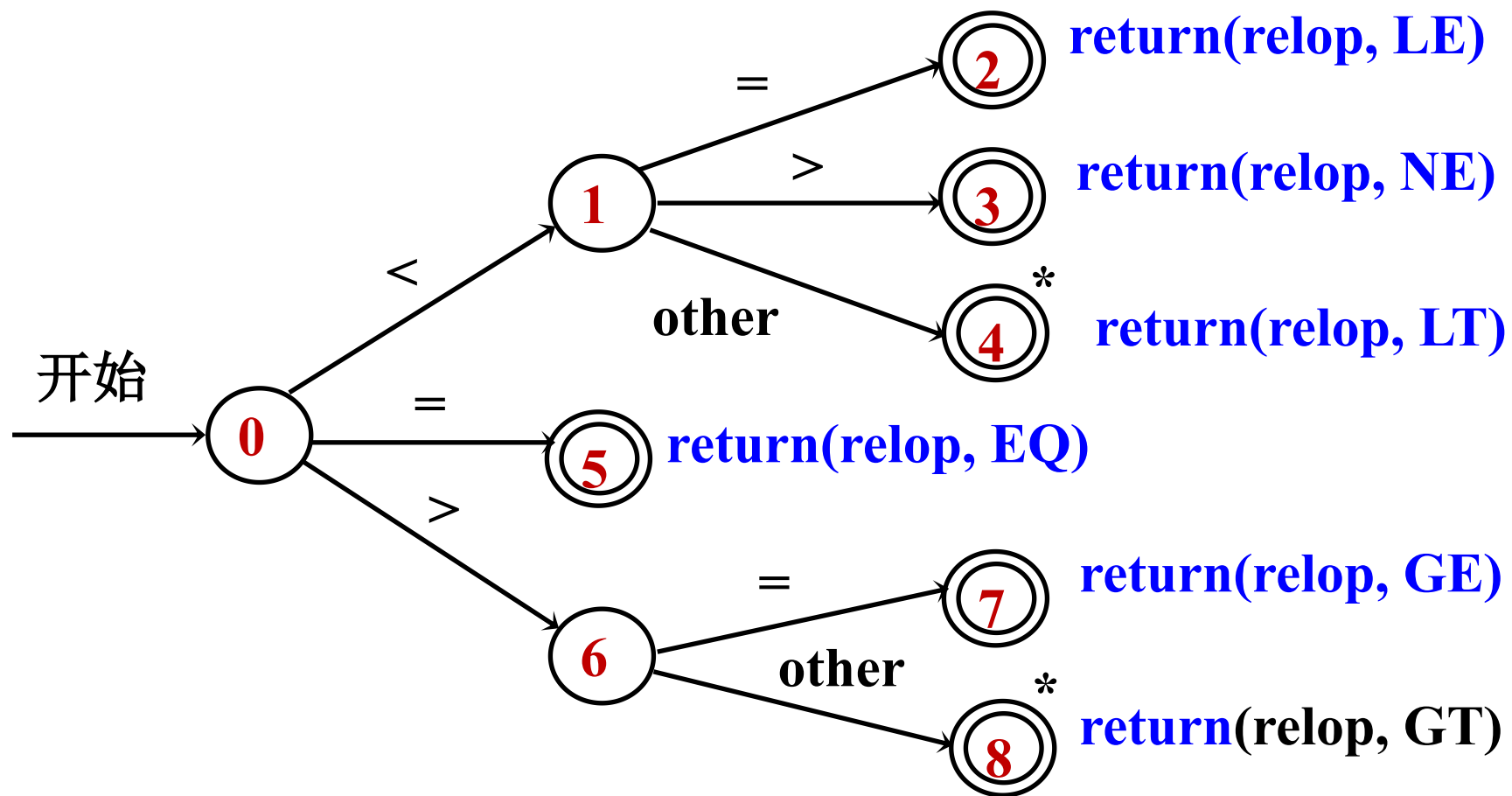
有限状态自动机 (FA)

- 有限自动机是依据某些输入而改变状态的机器或程序。可以用状态转换图来描述。
- 有限自动机是有向图这种通用结构的一个实例，在动态数据结构和高级搜索方面有许多应用。

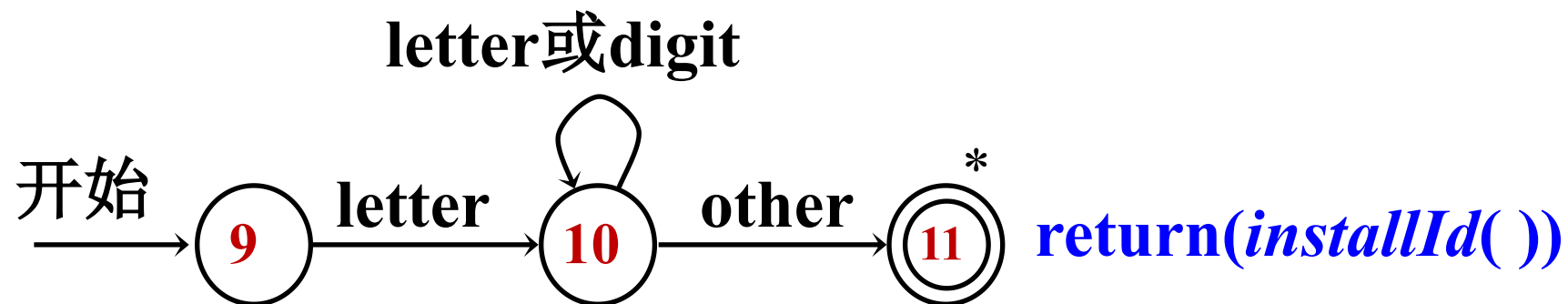
状态转换图

- 是一个有向图，**结点表示状态**，用**圆圈表示**；
- 用**双圆圈表示识别结束**；
- 用**带*的双圆圈**表示识别结束，**并且需要回退1个字符**；
- 结点之间可以用**有向边连接**；
- 有向边上标记影响状态改变的**可能输入的字符**；

关系算符的转换图



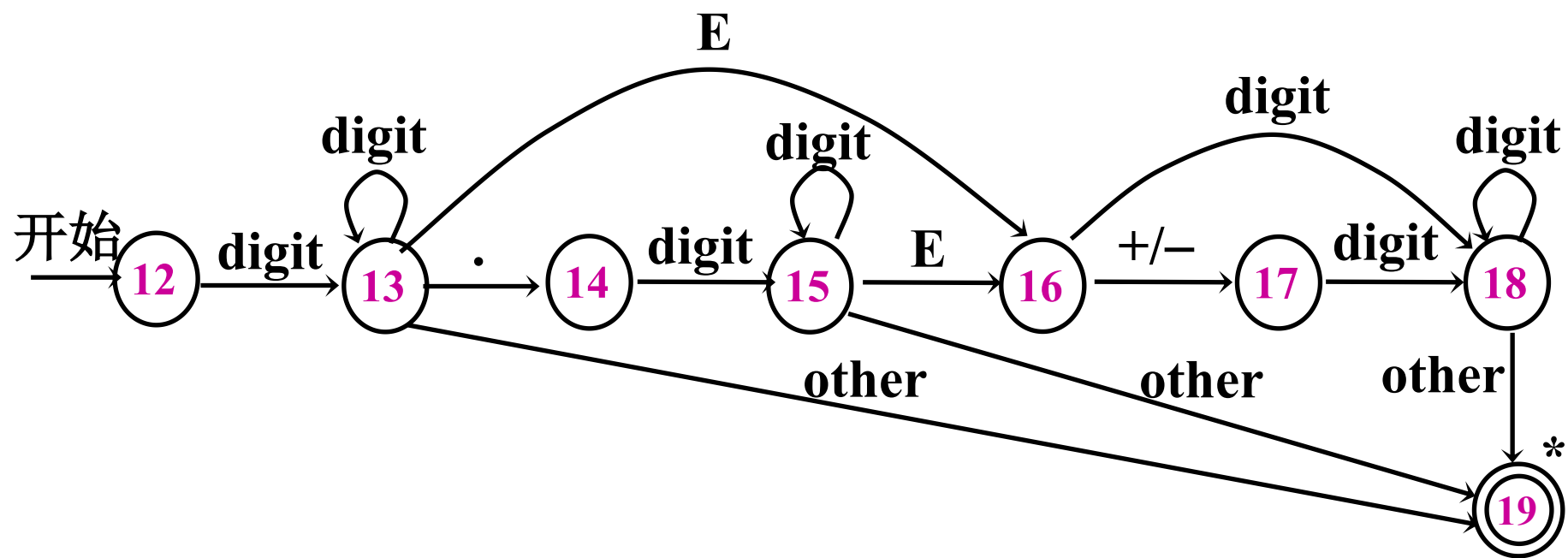
标识符和保留字的转换图



installId() 是一个函数，将识别的标识符插入符号表

无符号数的转换图

$\text{number} \rightarrow \text{digit}^+ (. \text{digit}^+)? (E (+ | -)? \text{digit}^+)?$

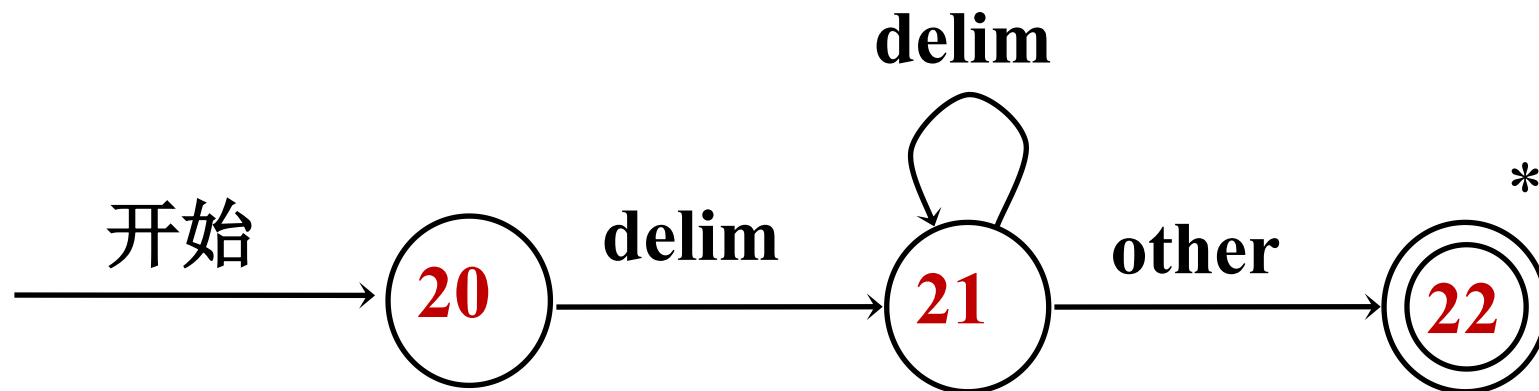


`return( number() )`

空格或空白行识别的转换图

delim $\rightarrow$ **blank** | **tab** | **newline**

ws $\rightarrow$ **delim**⁺



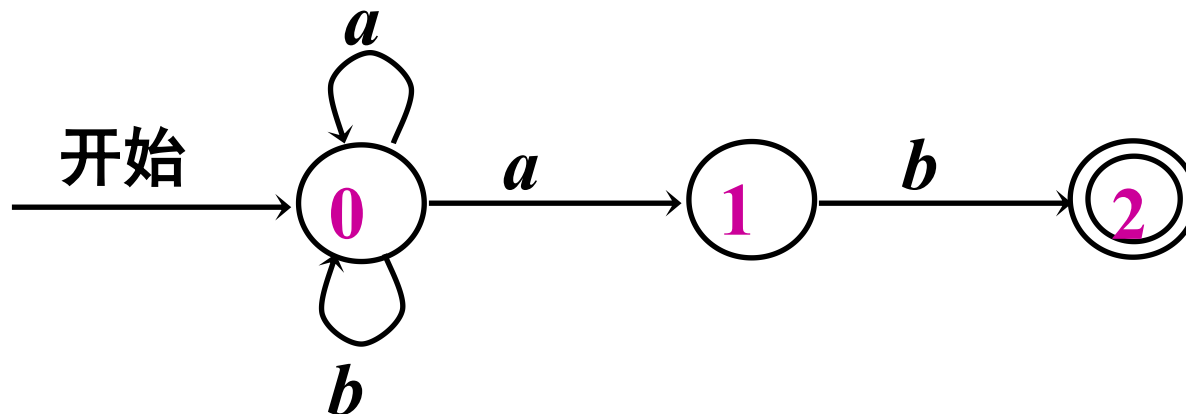
不确定的有限自动机（简称NFA）

NFA是一个数学模型，它包括：

- 1、有限状态集合 S
- 2、输入非空字母符号的集合 Σ
- 3、转换函数 $f: S \times (\Sigma \cup \{\epsilon\}) \rightarrow S$
- 4、状态 s_0 是开始状态
- 5、 $F \subseteq S$ 是接受状态集合

假设识别语言

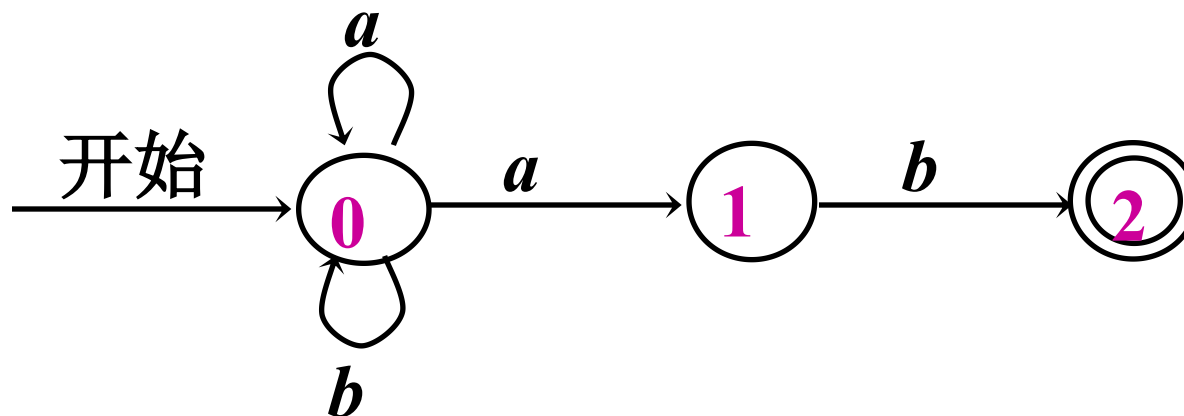
$(a|b)^*ab$ 的NFA



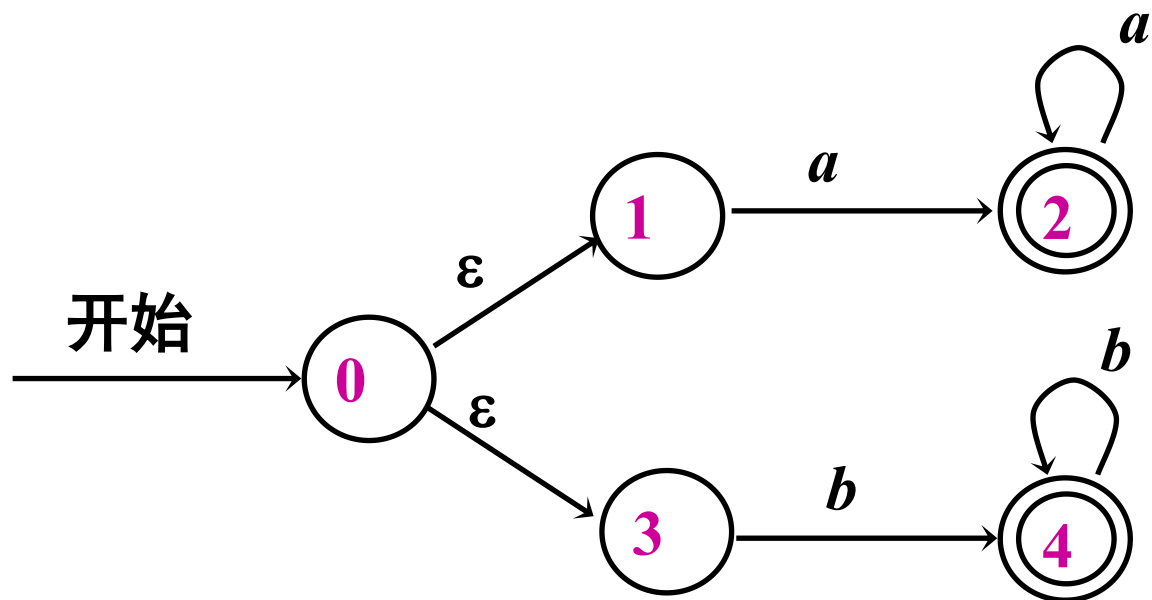
• NFA的转换表

状 态	输 入 符 号	
	a	b
0	$\{0, 1\}$	$\{0\}$
1	$\emptyset$	$\{2\}$
2	$\emptyset$	$\emptyset$

识别语言
 $(a|b)^*ab$ 的NFA



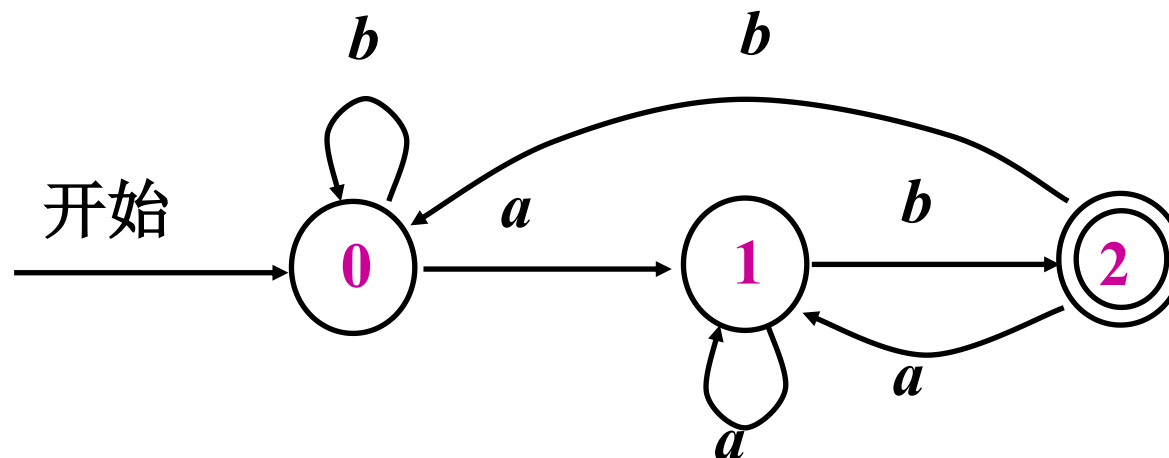
- 例: 识别 $aa^*|bb^*$ 的NFA



确定的有限自动机（简称DFA）

- 1、状态集合 S
- 2、输入符号集合 Σ
- 3、转换函数 $D : S \times \Sigma \rightarrow S$
- 4、唯一的初始状态 s_0
- 5、对于每个状态 s 和每个输入符号 a ，只有唯一标有 a 的边离开
- 6、 $F \subseteq S$ 是接受状态集合

识别语言
 $(a|b)^*ab$ 的DFA



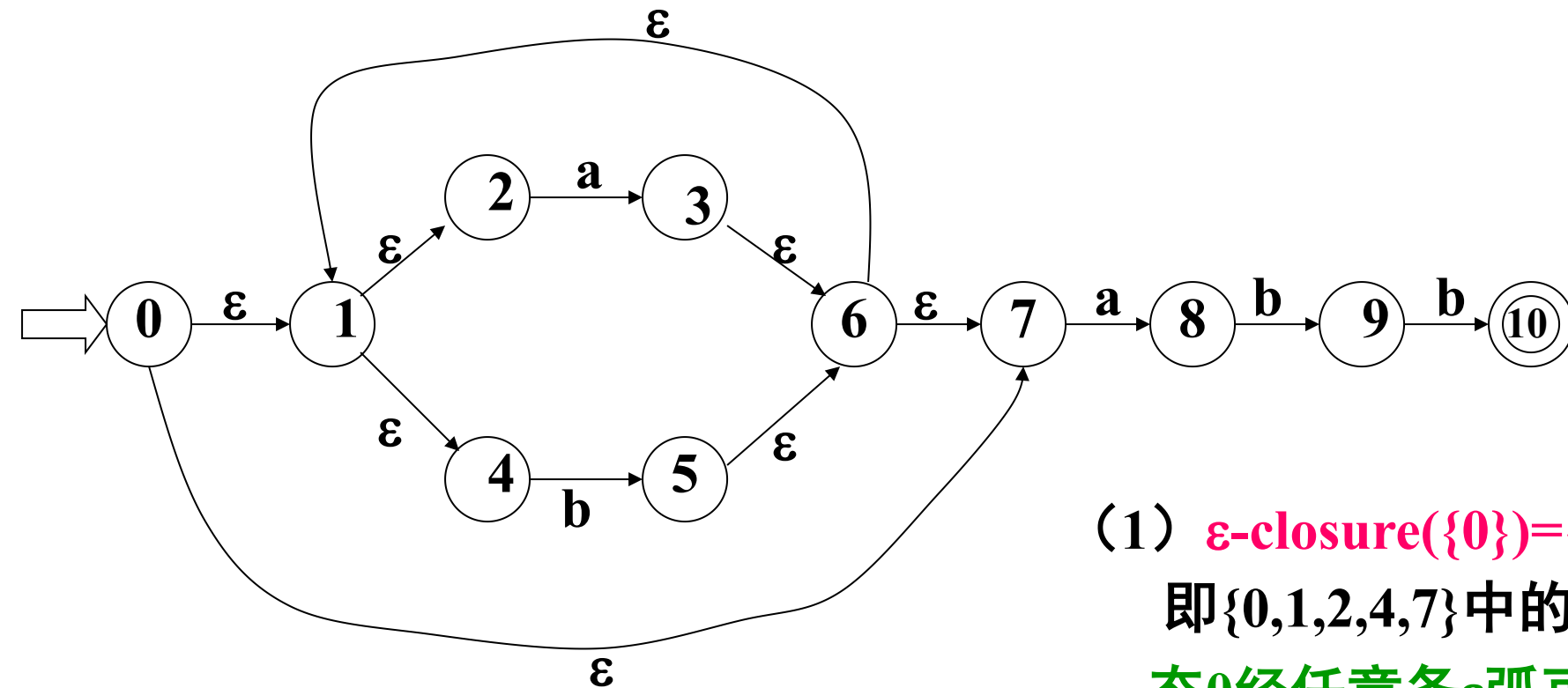
NFA到DFA的等价变换

- 使用NFA判定某个输入符号串的时候，可能出现不确定的情况：不知道下面选择那个状态。如果选择不好，该输入符号串可能不能到达终止状态。但是，我们不能说该输入符号串不能被该NFA接受。
- 如果通过尝试的方法，不断试探来确定输入符号串是否可被接受，那么判定的效率将降低。解决的方法是将NFA转换为等价的DFA。
- 介绍一种称为子集法的算法实现NFA→DFA的转换

NFA转换到DFA的相关运算

- ① 状态集 I 的 ϵ -闭包，表示为 $\epsilon\text{-closure}(I)$ ，是指从 I 中任意状态出发，通过空转移（ ϵ -转移）可以到达的所有状态的集合。计算 ϵ -闭包的算法通常使用深度优先搜索（DFS）来实现。
- ② 状态集 I 的 a 弧转移，表示为 $\text{move}(I,a)$ ，记为状态集 J ，则 J 表示从 I 中的某一状态经过一条 a 弧所到达的状态集。

NFA到DFA的变换



(1) $\epsilon\text{-closure}(\{0\}) = \{0, 1, 2, 4, 7\}$

即 $\{0, 1, 2, 4, 7\}$ 中的任一状态都是从状态0经任意条 ϵ 弧可到达的状态

(2) 令 $\{0, 1, 2, 4, 7\} = A$, 则 $\text{move}(A, a) = \{3, 8\}$

(3) $\epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$

构造NFA N状态K的子集的算法

假定所构造的子集族为T，即 $T=\{T_1, T_2, \dots, T_i\}$ 其中 $T_1, T_2, \dots, T_i$ 为状态S的子集。

① 开始，令 $\epsilon\text{-closure}(s_0)$ 为T中唯一成员，并且它是未被标记的。

② While(T中有未被标记的子集 T_i)do

{标记 T_i ;

for 每个输入字母a do

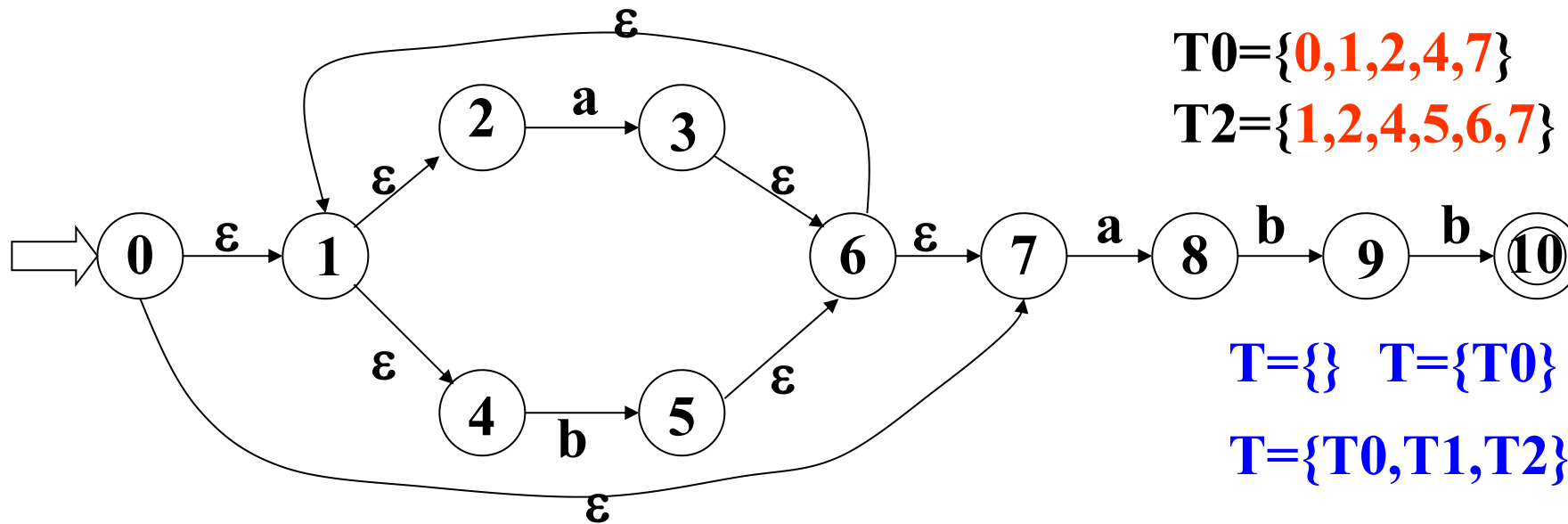
{ $U = \epsilon\text{-closure}(\text{Move}(T_i, a))$;

if U不在T中 then

将U做为未被标记的子集加在T中

}

}



$T0=\{0,1,2,4,7\}$

$T1=\{1,2,3,4,6,7,8\}$

$T2=\{1,2,4,5,6,7\}$

$T3=\{1,2,4,5,6,7,9\}$

$T=\{\}$ $T=\{T0\}$ $T=\{T0,T1\}$

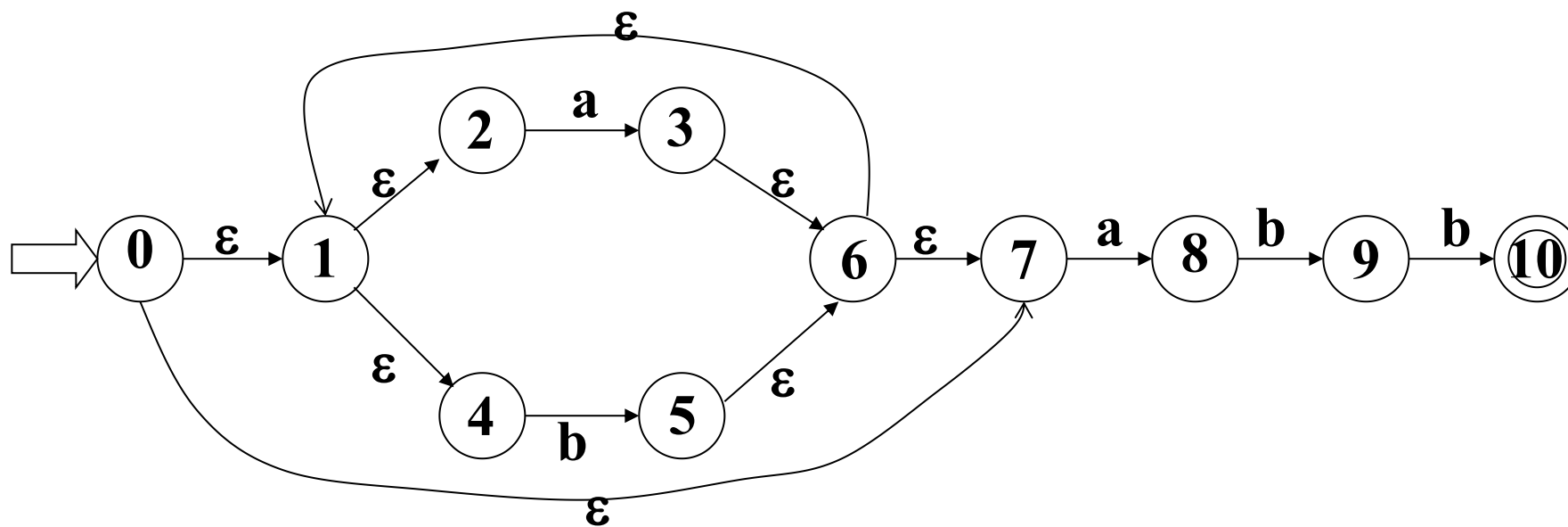
$T=\{T0,T1,T2\}$ $T=\{T0,T1,T2,T3\}$

1. 计算 $\epsilon\text{-closure}(\{0\}) = \{0,1,2,4,7\} = T0$, 没有在集合T中, 加入集合T中

2. 计算 $\epsilon\text{-closure}(\text{Move}(T0,a)) = \{1,2,3,4,6,7,8\} = T1$, 如没在集合T中, 加入T中; 计算 $\epsilon\text{-closure}(\text{Move}(T0,b)) = \{1,2,4,5,6,7\} = T2$, 如没在集合T中, 加入T中; 标记 $T0$ 。

3. 计算 $\epsilon\text{-closure}(\text{Move}(T1,a)) = \{1,2,3,4,6,7,8\} = T1$, 已在集合在T中;

计算 $\epsilon\text{-closure}(\text{Move}(T1,b)) = \{1,2,4,5,6,7,9\} = T3$, 没在集合在T中, 加入T中, 标记 $T1$ 。



$T_0 = \{0, 1, 2, 4, 7\}$

$T_1 = \{1, 2, 3, 4, 6, 7, 8\}$

$T_2 = \{1, 2, 4, 5, 6, 7\}$

$T_3 = \{1, 2, 4, 5, 6, 7, 9\}$

$T_4 = \{1, 2, 4, 5, 6, 7, 10\}$

$T = \{T_0, T_1, T_2, T_3\}$

$T = \{T_0, T_1, T_2, T_3, T_4\}$

4. 计算 ϵ -closure(Move(T_2, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$, 在 T 中已经存在 T_1 ;

计算 ϵ -closure(Move(T_2, b)) = $\{1, 2, 4, 5, 6, 7\} = T_2$, 在 T 中已经存在 T_2 ; 标记 T_2 。

5. 计算 ϵ -closure(Move(T_3, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$, 在 T 中已经存在 T_1 ;

计算 ϵ -closure(Move(T_3, b)) = $\{1, 2, 4, 5, 6, 7, 10\} = T_4$, 没在 T 中, 加入 T 中; 标记 T_3 。

6. 计算 ϵ -closure(Move(T_4, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$, 在 T 中已存在 T_1

计算 ϵ -closure(Move(T_4, b)) = $\{1, 2, 4, 5, 6, 7\} = T_2$, 在 T 中已存在 T_2 , 标记 T_4 。

1. 计算 ε -closure($\{0\}$) = $\{0, 1, 2, 4, 7\} = T_0$
2. 计算 ε -closure(Move(T_0, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$
 计算 ε -closure(Move(T_0, b)) = $\{1, 2, 4, 5, 6, 7\} = T_2$
3. 计算 ε -closure(Move(T_1, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$
 计算 ε -closure(Move(T_1, b)) = $\{1, 2, 4, 5, 6, 7, 9\} = T_3$
4. 计算 ε -closure(Move(T_2, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$
 计算 ε -closure(Move(T_2, b)) = $\{1, 2, 4, 5, 6, 7\} = T_2$
5. 计算 ε -closure(Move(T_3, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$
 计算 ε -closure(Move(T_3, b)) = $\{1, 2, 4, 5, 6, 7, 10\} = T_4$
6. 计算 ε -closure(Move(T_4, a)) = $\{1, 2, 3, 4, 6, 7, 8\} = T_1$
 计算 ε -closure(Move(T_4, b)) = $\{1, 2, 4, 5, 6, 7\} = T_2$

那么NFA N构造的DFA M为：

1. $S = \{T_0, T_1, T_2, T_3, T_4\}$

2. $\Sigma = \{a, b\}$

$D(T_0, a) = T_1$ $D(T_0, b) = T_2$

$D(T_1, a) = T_1$ $D(T_1, b) = T_3$

$D(T_2, a) = T_1$ $D(T_2, b) = T_2$

$D(T_3, a) = T_1$ $D(T_3, b) = T_4$

$D(T_4, a) = T_1$ $D(T_4, b) = T_2$

4. $S_0 = T_0$

5. $S_t = T_4$

包含原NFA
终态的T4加
到终态集合

将T0,T1,T2,T3,T4重新命名为A,B,C,D,E状态转换图变为

$D(T0,a)=T1$ $D(T0,b)=T2$

$D(T1,a)=T1$ $D(T1,b)=T3$

$D(T2,a)=T1$ $D(T2,b)=T2$

$D(T3,a)=T1$ $D(T3,b)=T4$

$D(T4,a)=T1$ $D(T4,b)=T2$



$D(A,a)=B$

$D(A,b)=C$

$D(B,a)=B$

$D(B,b)=D$

$D(C,a)=B$

$D(C,b)=C$

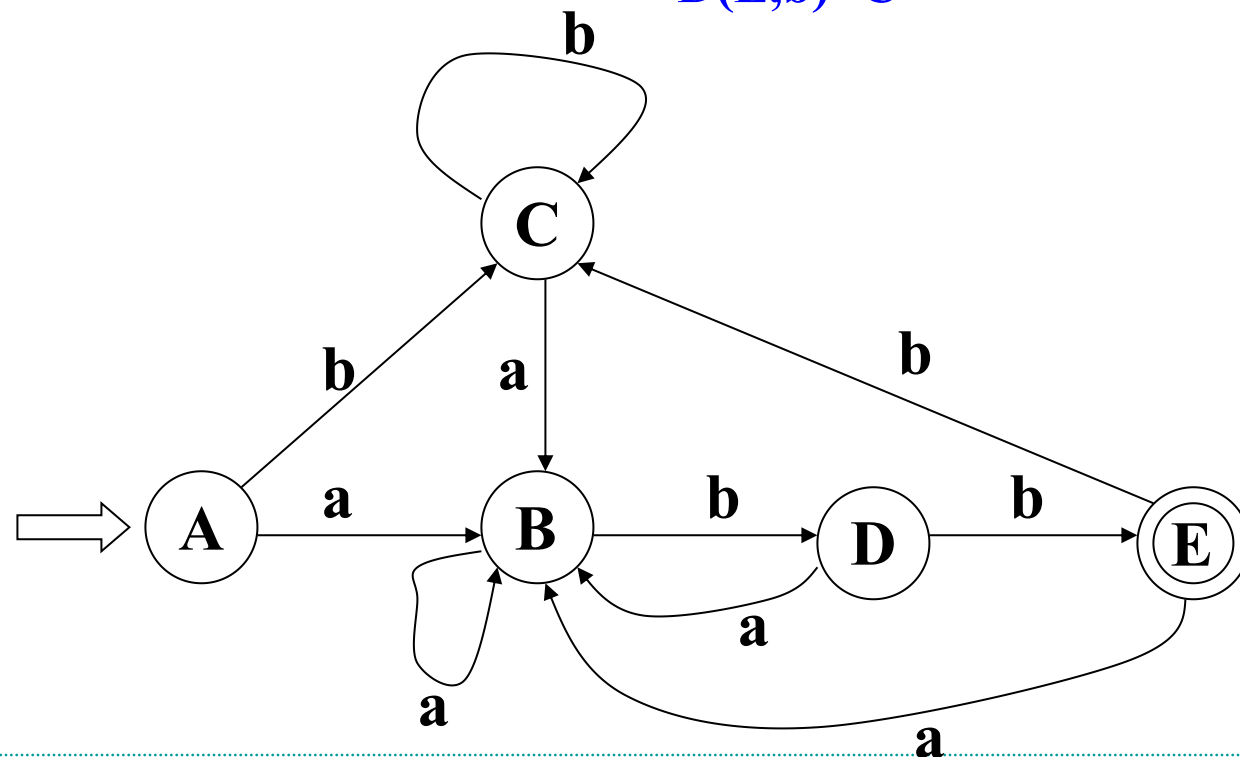
$D(D,a)=B$

$D(D,b)=E$

$D(E,a)=B$

$D(E,b)=C$

NFA状态	DFA状态	输入符号	
		<i>a</i>	<i>b</i>
$\{0,1,2,4,7\}$	<i>A</i>	<i>B</i>	<i>C</i>
$\{1,2,3,4,6,7,8\}$	<i>B</i>	<i>B</i>	<i>D</i>
$\{1,2,4,5,6,7\}$	<i>C</i>	<i>B</i>	<i>C</i>
$\{1,2,4,5,6,7,9\}$	<i>D</i>	<i>B</i>	<i>E</i>
$\{1,2,4,5,6,7,10\}$	<i>E</i>	<i>B</i>	<i>C</i>



确定有限自动机（DFA）最小化

- 确定有限自动机（DFA）最小化是指将一个给定的DFA转换为等价的具有最少状态数的DFA的过程。
- 最小化DFA的主要目的是简化自动机的结构，减少状态数，提高匹配效率。

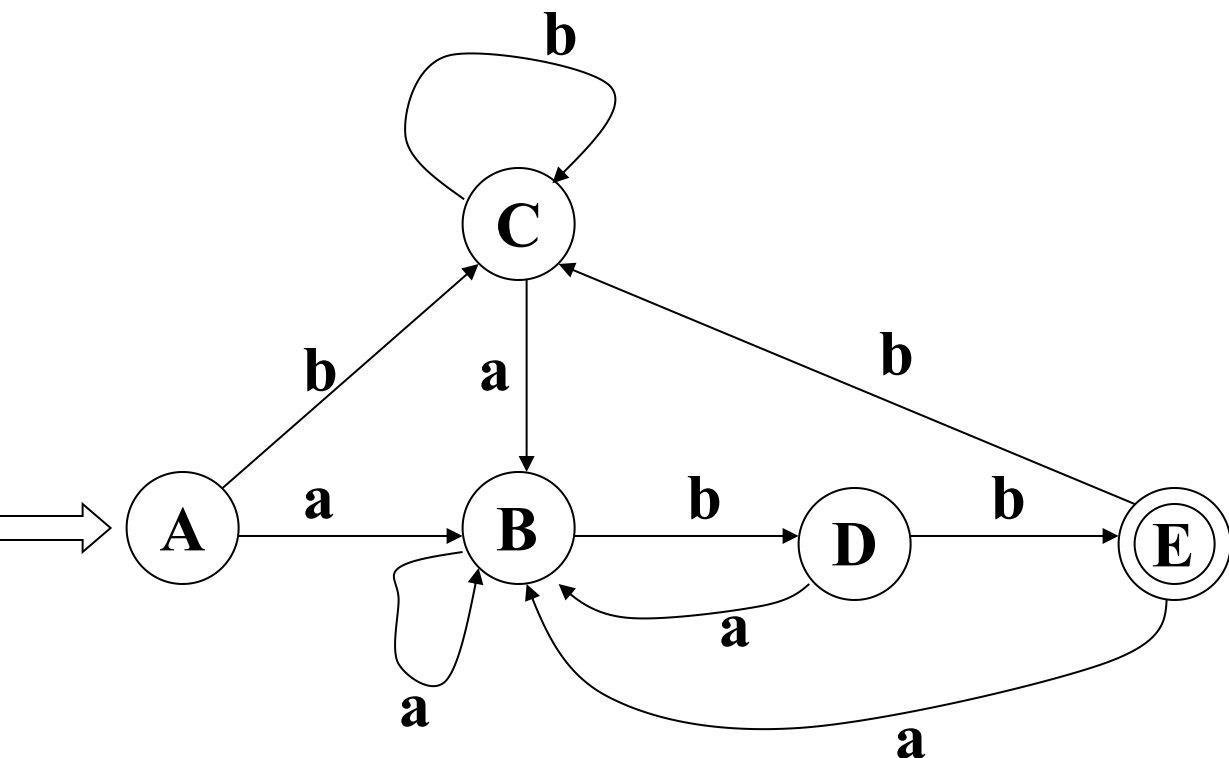
DFA最小化---Hopcroft算法

- Hopcroft算法是一种经典的确定有限自动机（DFA）最小化算法，它基于等价状态划分法，能够高效地将一个给定的 DFA 转换为具有最少状态数的等价 DFA。
- 对于每一对不同的状态 (p,q) ，如果 (p,q) 在任意输入符号下转移到相同的状态属于相同等价类，则将 (p,q) 划分到同类中；如果 (p,q) 在任意输入符号下转移到的状态分别属于不同的等价类，则将 (p,q) 划分到不同的等价类中。

Hopcroft算法的详细描述

- ✓ 步骤1：将 DFA 的状态划分为两类：接受状态和非接受状态。将接受状态和非接受状态分别构成两个初始等价类。
- ✓ 步骤2：重复以下步骤，直到不能再细分为止：
 - ① 对于每一对不同的状态 (p,q) ，以及 DFA 的每个输入符号 a ：计算状态 p 和 q 经过输入符号 a 转移到的状态分别为 r 和 s 。
 - ② 如果 r 和 s 不属于同一等价类，则将 (p,q) 划分到不同的等价类中。
 - ③ 检查是否有状态发生了划分，如果有则更新等价类划分，继续下一轮迭代。
- ✓ 步骤3：对分划完成后的每个状态子集，合并为一个状态，同时合并状态转移。

Hopcroft算法示例



第1步：将自动机的状态划分两个子集形成等价划分：

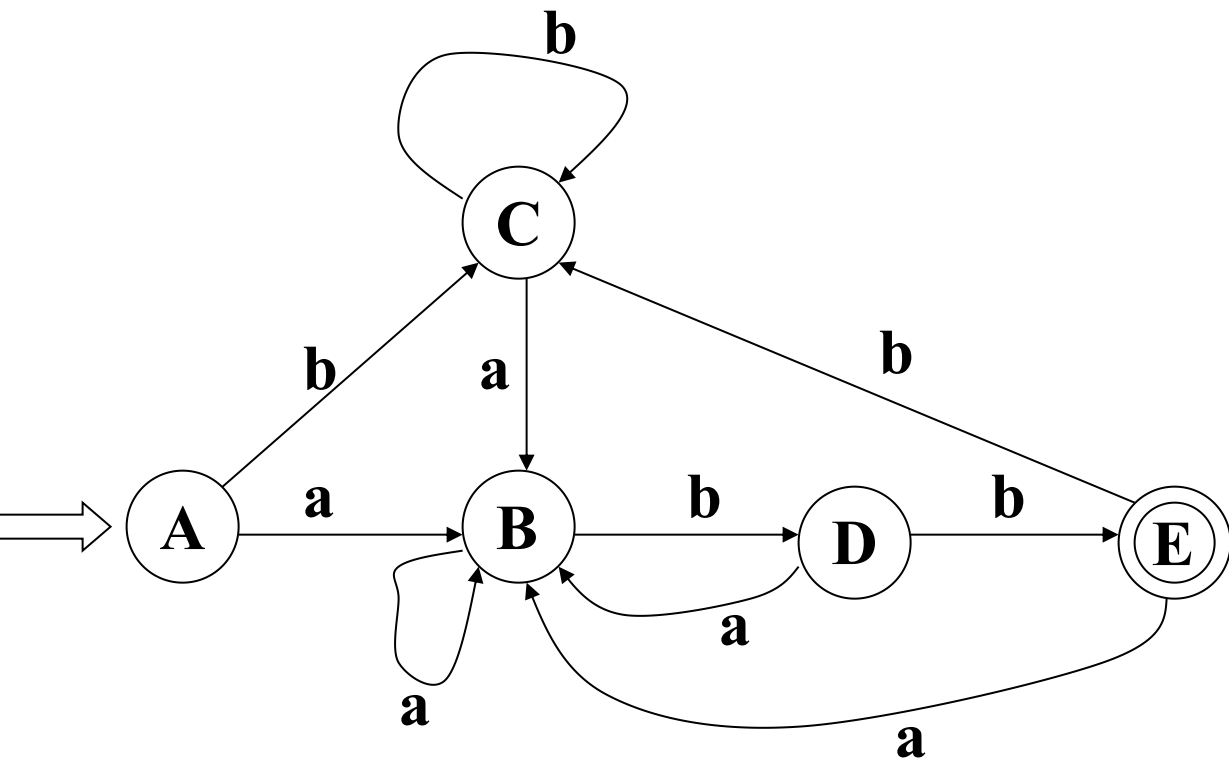
$G1=\{A,B,C,D\}$, $G2=\{E\}$

第2步：由于G2只有E，不可能再分；考察G1的每个状态ABCD,输入字符a, 都转移到子集G1；输入字符b, 状态ABC转移到G1, D转移到G2, 此时, G1划分为子集 $G3=\{A,B,C\}$, $G4=\{D\}$;

第3步：由于G4只有D, 不可能再分；考察G3的每个状态ABC,输入字符a, 都转移到子集G3；输入字符b, 状态AC转移到G3, B转移到G4, 此时, G3划分为子集 $G5=\{A,C\}$, $G6=\{B\}$;

第4步：由于G6只有B, 不可能再分；考察G5的每个状态AC,输入字符a, 都转移到子集G6；输入字符b, 都转移到G5, 因此, 停止划分。

Hopcroft算法示例（续）



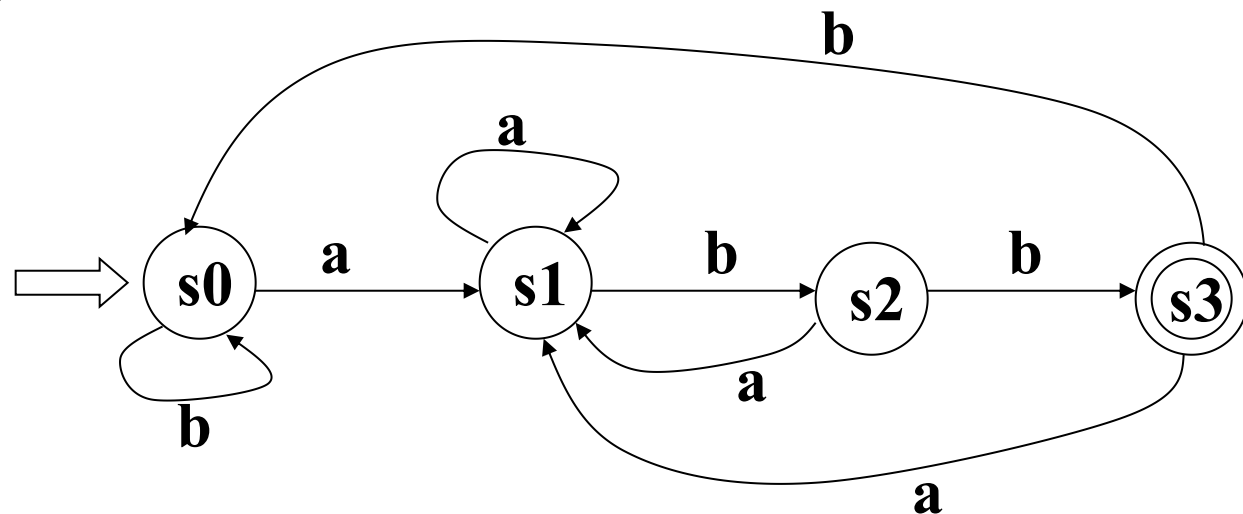
第5步：将自动机的状态划分四个子集：

$G4=\{D\}$, $G5=\{A,C\}$, $G6=\{B\}$, $G2=\{E\}$;

第6步：将状态集重新命名：

$s0=\{A,C\}$, $s1=\{B\}$, $s2=\{D\}$, $s3=\{E\}$;

重新画出状态转换图如下：



正则表达式与有限状态自动机的关系

- 正则表达式的特点：
 - ① 正则表达式提供了丰富的语法和操作符，可以描述各种复杂的模式；
 - ② 用较短的表达式就能描述复杂的匹配规则，简化了模式匹配的过程；
 - ③ 适用于多种编程语言和工具，是一种通用的字符串匹配工具。
- 有限状态自动机的特点：
 - ① 确定有限自动机在处理字符串匹配时具有确定性，不会出现歧义。
 - ② 提供了一种形式化的方式来描述和处理字符串匹配问题，直观易懂，便于理论分析和算法设计。
- 在字符串识别正则表达式与有限状态自动机是等价的

由正则式转换成DFA---Thompson构造法

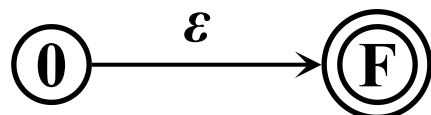
正则式

转换系统

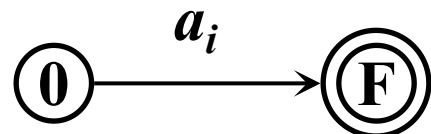
$\emptyset$



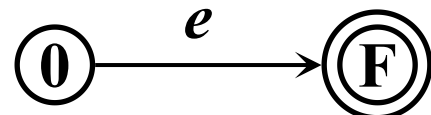
ε



a_i



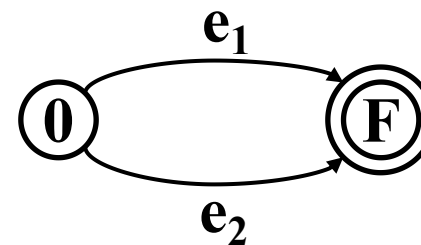
e



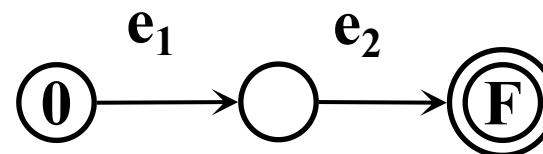
正则式

转换系统

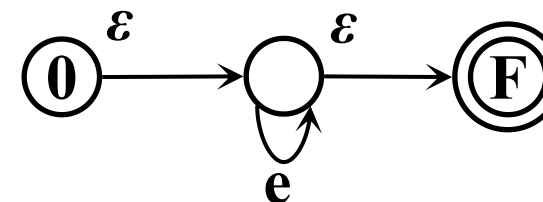
$e_1 \mid e_2$



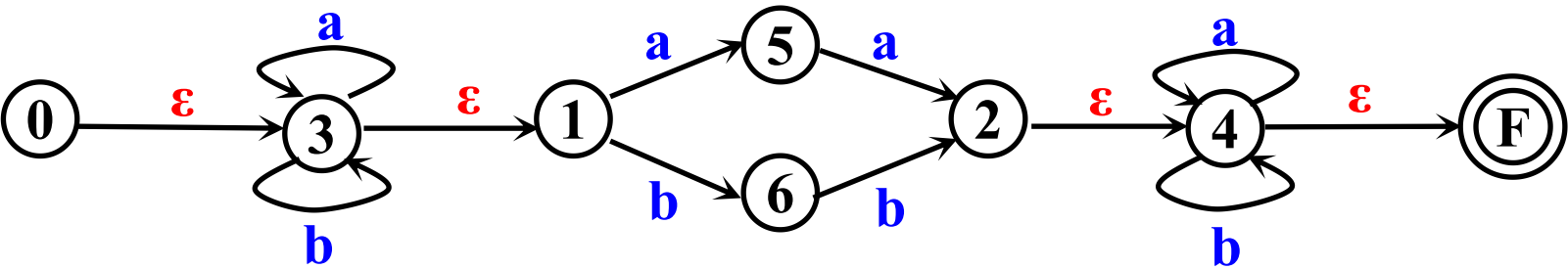
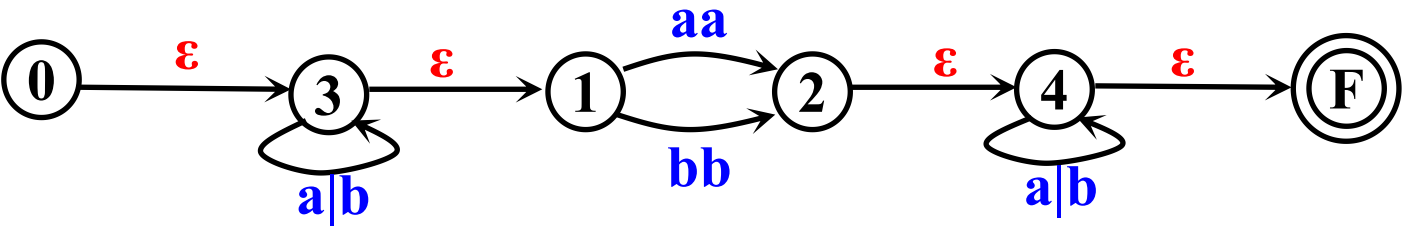
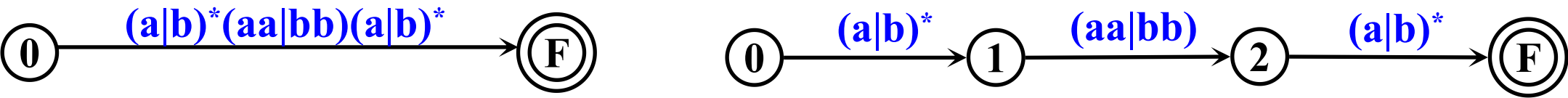
$e_1 e_2$



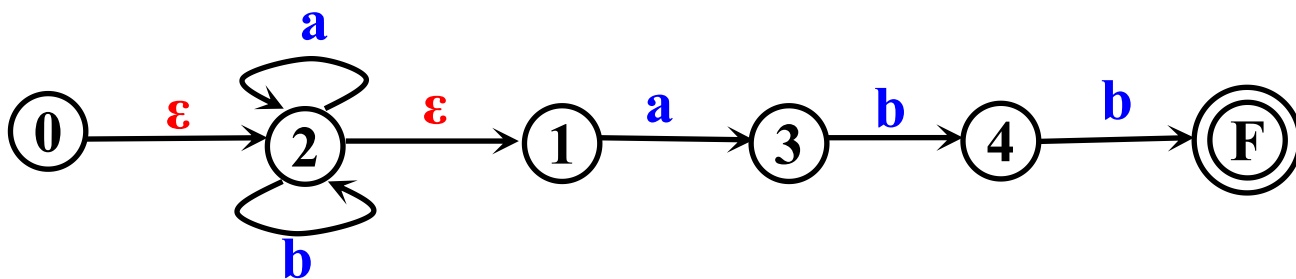
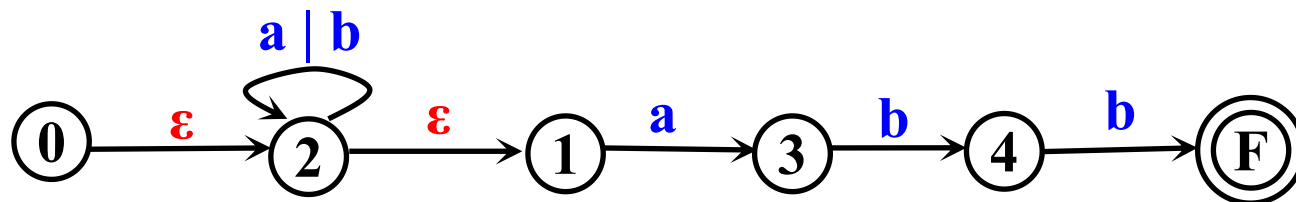
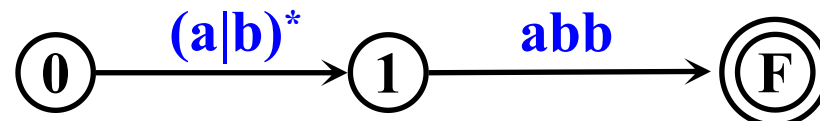
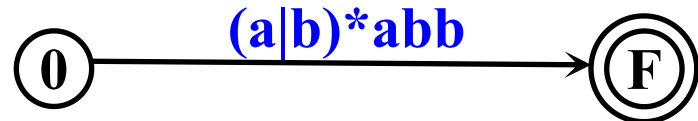
e^*



由正则式转换成DFA示例 $(a|b)^*(aa|bb)(a|b)^*$



将正则式 $(a|b)^*abb$ 转换成DFA示例



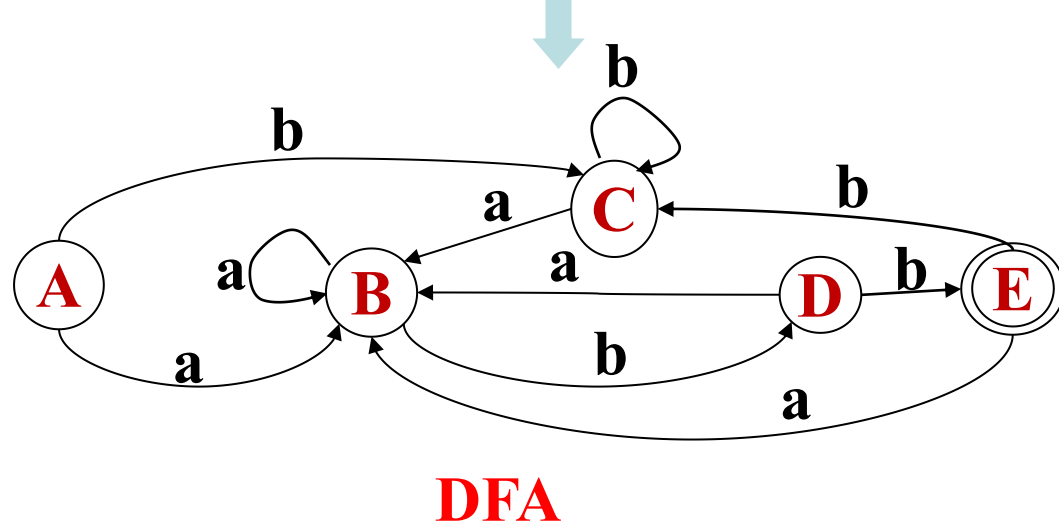
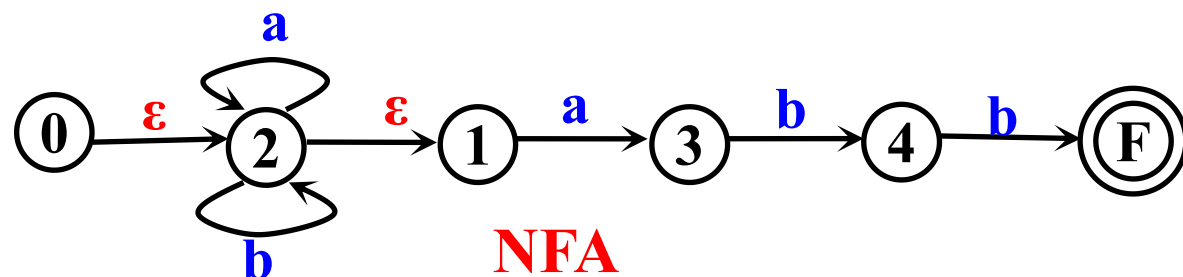
词法分析器的实现方法

- **手写词法分析器**：根据编程语言的词法规则，即按照有限自动机的状态转换表手工编写词法分析器的程序代码。精干运行效率高，实现难度较大。如：GCC，LLMV
- **使用词法分析器生成工具**：通过编写词法规则描述文件，使用工具自动生成词法分析器的代码。常见的词法分析器生成工具有：
 - ① Lex / Flex：用于生成C语言的词法分析器
 - ② JLex / JFlex：用于生成Java语言的词法分析器
 - ③ ANTLR：支持多种编程语言的词法分析器生成

构造词法分析器的一般步骤

- ① 用正式式对程序设计语言单词模式进行描述；
- ② 使用Thompson构造法，根据正则表达式构建对应的NFA。
- ③ 使用子集构造法（subset construction）将NFA转换为DFA；
- ④ 简化DFA，使其状态数最少；
- ⑤ 从化简后的DFA构造词法分析程序。

将NFA转换为DFA



$$\epsilon\text{-closure}(0)=\{0,1,2\}=A$$

$$\epsilon\text{-closure}(\text{move}(A,a))=\{1,2,3\}=B$$

$$\epsilon\text{-closure}(\text{move}(A,b))=\{1,2\}=C$$

$$\epsilon\text{-closure}(\text{move}(B,a))=\{1,2,3\}=B$$

$$\epsilon\text{-closure}(\text{move}(B,b))=\{1,2,4\}=D$$

$$\epsilon\text{-closure}(\text{move}(C,a))=\{1,2,3\}=B$$

$$\epsilon\text{-closure}(\text{move}(C,b))=\{1,2\}=C$$

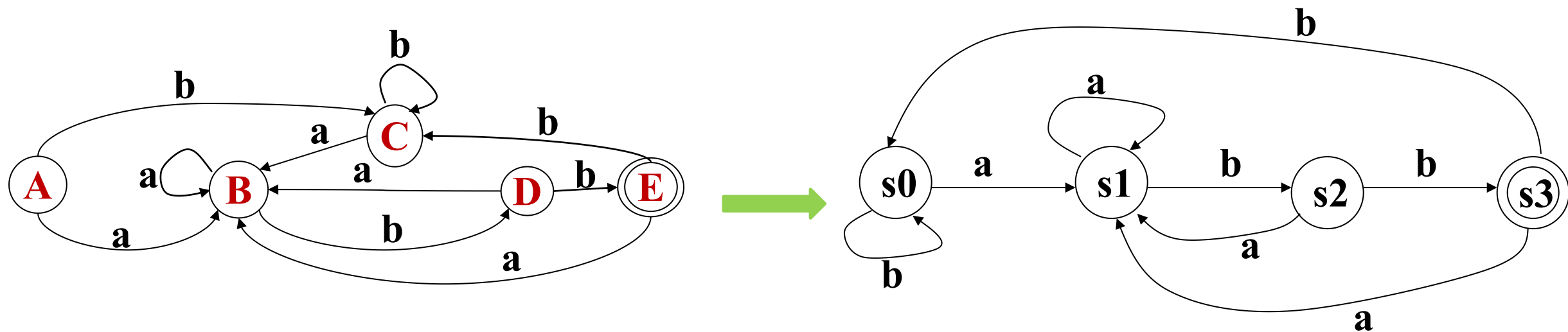
$$\epsilon\text{-closure}(\text{move}(D,a))=\{1,2,3\}=B$$

$$\epsilon\text{-closure}(\text{move}(D,b))=\{1,2,F\}=E$$

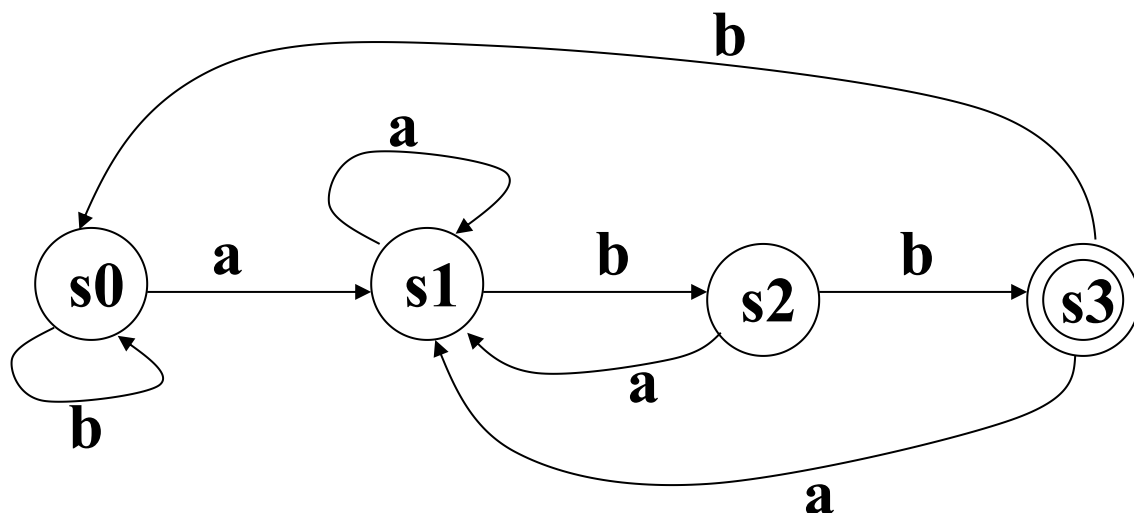
$$\epsilon\text{-closure}(\text{move}(E,a))=\{1,2,3\}=B$$

$$\epsilon\text{-closure}(\text{move}(E,b))=\{1,2\}=C$$

化简DFA



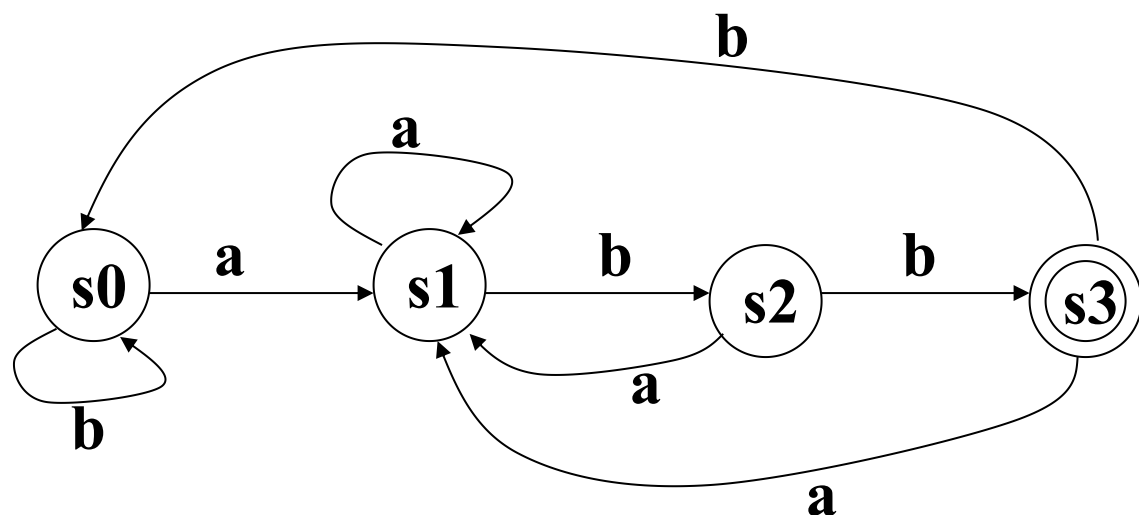
$(a|b)^*abb$ 等价的DFA如下:



- 可以根据DFA设计词法分析程序，右边是对应的简易的词法分析程序，从输入的字符串中识别出符合 $(a|b)^*abb$ 模式的词法单元；
- 程序核心部分采用while循环和switch分支处理识别符合正则表达式的词法单元，switch的case分支对应DFA的状态，DFA的状态变化在程序中用state来记录。

```
1  #include <stdio.h>
2  #include <string.h>
3  int main()
4  {
5      char input[100];
6      int state = 0;
7      int i = 0;
8      int start_index = 0;
9      int j = 0;
10     printf("Enter input string: ");
11     gets(input);
12     while (input[i] != '\0')
13     {
14         switch (state)
15         {
16             case 0:
17                 if (input[i] == 'a')
18                 {
19                     state = 1;
20                 }
21                 else if (input[i] == 'b')
22                 {
23                     state = 2;
24                 }
25                 else
26                 {
27                     state = -1; /* Invalid input*/
28                 }
29                 break;
```

$(a|b)^*abb$ 正则表达式的模式串识别



- 右边程序片段是完成**状态1**和**状态2**的处理
- **state=-1**表明输入的正则表达式不能识别的字符，此处**可以是一些错误处理代码**。

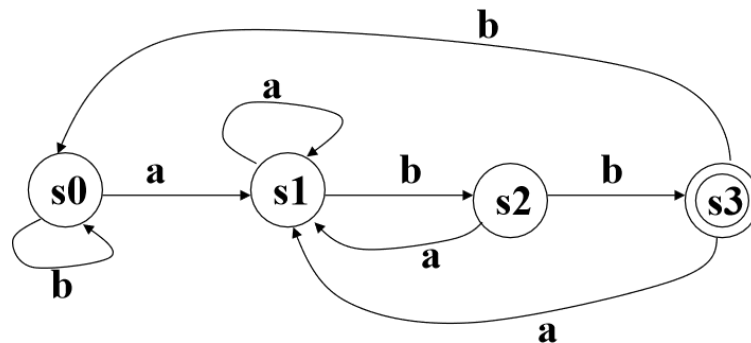
```
30 case 1:
31     if (input[i] == 'a')
32     {
33         state = 1;
34     }
35     else if (input[i] == 'b')
36     {
37         state = 2;
38     }
39     else
40     {
41         state = -1; /* Invalid input*/
42     }
43     break;
44 case 2:
45     if (input[i] == 'a')
46     {
47         state = 1;
48     }
49     else if (input[i] == 'b')
50     {
51         state = 3;
52     }
53     else
54     {
55         state = -1; /*Invalid input*/
56     }
57     break;
```



```

58 case 3:
59     if (input[i] == 'a')
60     {
61         state = 0;
62     }
63     else if (input[i] == 'b')
64     {
65         state = 1;
66     }
67     else if (input[i] == ' ')
68     {
69         for (j = start_index; j <= i; j++)
70         {
71             printf("%c", input[j]);
72         }
73         printf("\n");
74     }
75     else
76     {
77         state = -1; /*Invalid input*/
78     }
79     break;
80 default:
81     break;
82 }
83 if (input[i] == ' ')
84 {
85     state = 0;
86     start_index = i + 1;
87 }
88 i++;
89 }

```



```

90
91     if (state == 3)
92     {
93         for (j = start_index; j <= i; j++)
94         {
95             printf("%c", input[j]);
96         }
97         printf("\n");
98     }
99
100     return 0;
101 }

```

本示例假定空格作为词法单元（单词）的分隔符，如果state=3且当前字符为空格时，此时已经识别了一个符合正则表达式的词法单元，使用循环语句打印输出，否则丢弃该单词，进行下一个单词的识别。

```

C:\Windows\system32\cmd.e:  x  +  v
Enter input string: aaabbabc abbabb ababbabb baababbbaa abbabb
abbabb
ababbabb
abbabb
请按任意键继续 . . .

```

词法分析器的实现方法

- **手写词法分析器**：按照正则表达式和有限自动机手工编写词法分析器的程序，基本方法类似前面的示例，代码精干运行效率高，实现难度较大。
- **使用词法分析器生成工具**：通过编写词法规则描述文件，使用工具自动生成词法分析器的代码。常见的词法分析器生成工具如下，将在实验课介绍Flex工具：
 - ① **Lex / Flex**：用于生成C语言的词法分析器
 - ② **JLex / JFlex**：用于生成Java语言的词法分析器
 - ③ **ANTLR**：支持多种编程语言的词法分析器生成

一个简易词法分析器的实现（1）

```
#include <stdio.h>
#include <ctype.h>
#include <stdbool.h>
#include <string.h>
```

// 状态枚举

```
typedef enum {
    START,
    IN_COMMENT,
    IN_NUM,
    IN_ID,
    IN_ASSIGN,
    IN_SLASH,
    IN_STRING,
    DONE
} Token;
```

```
// 判断是否为字母或下划线
bool isAlphaOrUnderscore(char c) {
    return isalpha(c) || c == '_';
}
```

// 词法分析器

```
void lexer(const char *code) {
    Token token = START;
    char currentChar;
    int pos = 0;
    while (token != DONE) {
        currentChar = code[pos++];
```

```
switch (token) {
    case START:
        if (currentChar == '/') {
            token = IN_SLASH;
        } else if (isdigit(currentChar)) {
            token = IN_NUM;
            printf("NUM: %c", currentChar);
        } else if (isAlphaOrUnderscore(currentChar)) {
            token = IN_ID;
            printf("ID: %c", currentChar);
        } else if (currentChar == '=') {
            token = IN_ASSIGN;
            printf("ASSIGN: %c", currentChar);
        } else if (currentChar == '"') {
            token = IN_STRING;
            printf("STRING: ");
        } else if (currentChar == '\0') token = DONE;
        break;
```

一个简易词法分析器的实现（2）

case IN_SLASH:

```
if (currentChar == '*') {  
    token = IN_COMMENT;  
} else {  
    printf("DIVIDE: \n");  
    token = START;  
    pos--; // 回退一个字符  
}  
break;
```

case IN_COMMENT:

```
if (currentChar == '*' && code[pos] == '/') {  
    token = START;  
    pos++; // 跳过 '/'  
} else if (currentChar == '\0') {  
    token = DONE;  
}  
break;
```

case IN_NUM:

```
if (isdigit(currentChar)) {  
    printf("%c", currentChar);  
} else {  
    token = START;  
    printf("\n");  
    pos--; // 回退一个字符  
}  
break;
```

case IN_ID:

```
if (isAlphaOrUnderscore(currentChar) ||  
    isdigit(currentChar)) {  
    printf("%c", currentChar);  
} else {  
    token = START;  
    printf("\n");  
    pos--; // 回退一个字符  
} break;
```

一个简易词法分析器的实现（3）

case IN_ASSIGN:

```
if (currentChar == '=') {  
    printf("%c\n", currentChar);  
} else {  
    printf("\n");  
    pos--; // 回退一个字符  
}  
token = START;  
break;
```

case IN_STRING:

```
if (currentChar == '"') {  
    printf("\n");  
    token = START;  
} else if (currentChar == '\0') {  
    token = DONE;  
} else {  
    printf("%c", currentChar);  
}  
break;  
default:  
break;  
}  
}  
}
```

一个简易词法分析器的实现（3）

```
int main() {  
    const char *code = "int a = 42;\n/* This is a comment *\nb = a + 2;\nchar *str = \"Hello, World!\";\n";  
    lexer(code);  
    return 0;  
}
```

词法分析器可以识别数字、标识符、赋值符号、除法符号、字符串和多行注释。
请注意，这个实现仍然相对简单，仅用于演示目的。实际的C语言词法分析器会更加复杂，需要处理更多的状态和符号。

运行输出的结果

ID: int

ID: a

ASSIGN: =

NUM: 42

ID: b

ASSIGN: =

ID: a

NUM: 2

ID: char

ID: str

ASSIGN: =

STRING: Hello, World!

```
ID: int
ID: a
ASSIGN: =
NUM: 42
ID: b
ASSIGN: =
ID: a
NUM: 2
ID: char
ID: str
ASSIGN: =
STRING: Hello, World!
```

词法分析相关的作业

- ① 请写出一个正则表达式，匹配所有由 0 和 1 组成的偶数个字符的字符串。
- ② 给定正则表达式 $(a|b)(aa|bb)^*(a|b)$ ，请画出对应的非确定有限自动机（NFA），并转换为确定有限自动机（DFA），并最小化DFA；请编程实现从字符串中提取符合该正则表达式的C语言程序。
- ③ 给定一个确定有限自动机（DFA），如何将其转换为等价的正则表达式？
- ④ 给定一个确定有限自动机（DFA），如何判断它是否已经是最小化的状态？